BCR-6605 – Configuration update vs. active deviations

Benoit's 27-10 note, implemented. Two warnings, one confirmation popup, every deviation of the aircraft acknowledged on Continue.

What to do

☐	File	Package	Action
	`DeviationWarningUtils.java`	`com.eurocopter.mgs.presentation.utils`	**Create** – section 1
	`OkCreateUpdateListener.java`	`com.eurocopter.mgs.presentation.aircraft.details`	**Replace** – section 2
	message bundle	the one holding `MaxPointsPerHour`	**Add 5 keys** – section 3
	`AircraftDetailsComposite.java`	`...presentation.aircraft.details`	Nothing – warning already coded
	`ImportConfigurationComposite.java`	`...aircraft.management.composite`	Nothing – warning already coded

Why these two files. Four classes already call `DeviationWarningUtils` but the class does not exist anywhere – four compile errors. And `OkCreateUpdateListener` acknowledged **after** `updateAircraft`: the update builds a new indicator configuration set, so the active-deviation lookup finds nothing by then and the deviations stay open in the database for ever. The ids are now read before the update and acknowledged after it. The guard also covers the **usage** file, as Benoit wrote, and the default button is **Cancel** so pressing Enter cannot wipe the deviations.

1 – DeviationWarningUtils.java (new file)

Package: `com.eurocopter.mgs.presentation.utils` | beside `RtbUtils.java`

```
/**************************************/
/** Copyright Airbus Helicopters 2019 **/
/**************************************/
package com.eurocopter.mgs.presentation.utils;

import java.util.ArrayList;
import java.util.List;

import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
import org.osgi.framework.BundleContext;
import org.osgi.framework.FrameworkUtil;

import com.eurocopter.mgs.business.health.api.IHealthCommonBusiness;
import com.eurocopter.mgs.presentation.common.MgsSession;
import com.eurocopter.mgs.service.health.common.dto.IndicatorDeviationDto;

/**
 * BCR-6605 deviation helpers for the aircraft configuration screens.
 * <P>
 * Importing a new health or usage configuration invalidates every deviation set on the aircraft, so
 * the "Update Aircraft" and "Import Configurations" popups warn the user, and the confirmation popup
 * acknowledges them before the new configuration becomes the active one.
 * <P>
 * Every method is defensive: if the service cannot be reached the warning is simply not shown and
 * nothing is acknowledged, so a server problem can never block the aircraft update.
 */
public final class DeviationWarningUtils {

   private static final Logger LOGGER = LogManager.getLogger(DeviationWarningUtils.class);

   /**
    * Utility class, not instantiable.
    */
   private DeviationWarningUtils() {
      super();
   }

   /**
    * @return the health common business service, never null unless the bundle is not started
    */
   private static IHealthCommonBusiness getHealthBusiness() {
      final BundleContext healthBundleContext = FrameworkUtil.getBundle(IHealthCommonBusiness.class).getBundleContext();
      return healthBundleContext.getService(healthBundleContext.getServiceReference(IHealthCommonBusiness.class));
   }

   /**
    * The identifiers of the deviations currently active on the aircraft.
    * <P>
    * Read this BEFORE the configuration is updated: the update builds a new indicator configuration
    * set, and after it the "active deviation" lookup no longer returns the old rows.
    *
    * @param aircraftId
    *           the technical identifier of the aircraft, may be null
    * @return the active deviation identifiers, never null, empty when there is none
    */
   public static List<String> getActiveDeviationIds(final String aircraftId) {
      final List<String> ids = new ArrayList<>();
      if (aircraftId == null || aircraftId.isBlank()) {
         return ids;
      }
      try {
         for (IndicatorDeviationDto deviation : getHealthBusiness().findActiveDeviations(aircraftId)) {
            ids.add(deviation.getId());
         }
      } catch (Exception e) {
         LOGGER.warn("Could not read the active deviations of aircraft {}", aircraftId, e);
      }
      return ids;
   }

   /**
    * Whether the aircraft currently carries at least one active deviation.
    *
    * @param aircraftId
    *           the technical identifier of the aircraft, may be null
    * @return true when at least one deviation is active
    */
   public static boolean hasActiveDeviations(final String aircraftId) {
      return !getActiveDeviationIds(aircraftId).isEmpty();
   }

   /**
    * Acknowledge the given deviations: the end date is set to now, they leave the deviation list for
    * the deviation history, and a relearn is launched so the new configuration starts clean.
    *
    * @param aircraftId
    *           the technical identifier of the aircraft, may be null
    * @param deviationIds
    *           the deviations to acknowledge, may be null or empty
    */
   public static void acknowledge(final String aircraftId, final List<String> deviationIds) {
      if (aircraftId == null || deviationIds == null || deviationIds.isEmpty()) {
         return;
      }
      try {
         getHealthBusiness().acknowledgeDeviations(aircraftId, MgsSession.getInstance().getUser().getLoginName(), deviationIds, true);
         LOGGER.info("BCR-6605 {} deviation(s) acknowledged on aircraft {} because the configuration was updated.", deviationIds.size(), aircraftId);
      } catch (Exception e) {
         LOGGER.error("Could not acknowledge the deviations of aircraft {}", aircraftId, e);
      }
   }
}
```

2 – OkCreateUpdateListener.java (replace the whole file)

Package: `com.eurocopter.mgs.presentation.aircraft.details`

Built from your own source, so everything outside the BCR-6605 block is byte-identical.

```
package com.eurocopter.mgs.presentation.aircraft.details;

import java.text.MessageFormat;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
import org.eclipse.core.runtime.IStatus;
import org.eclipse.jface.dialogs.MessageDialog;
import org.eclipse.swt.events.SelectionAdapter;
import org.eclipse.swt.events.SelectionEvent;
import org.eclipse.ui.statushandlers.StatusManager;
import org.osgi.framework.BundleContext;
import org.osgi.framework.FrameworkUtil;

import com.eurocopter.mgs.app.Activator;
import com.eurocopter.mgs.business.aircraft.api.ConfigurationDto;
import com.eurocopter.mgs.business.aircraft.api.IAircraftBusiness;
import com.eurocopter.mgs.business.aircraft.api.IAircraftManagementBusiness;
import com.eurocopter.mgs.core.presentation.i18n.Messages;
import com.eurocopter.mgs.datamodel.aircraft.Aircraft;
import com.eurocopter.mgs.datamodel.aircraft.DataTransferType;
import com.eurocopter.mgs.datamodel.core.FeatureName;
import com.eurocopter.mgs.datamodel.flight.Flight;
import com.eurocopter.mgs.datamodel.rtb.MainRotorTuning;
import com.eurocopter.mgs.datamodel.rtb.MainTuningValue;
import com.eurocopter.mgs.datamodel.rtb.TailRotorTuning;
import com.eurocopter.mgs.datamodel.rtb.TailTuningValue;
import com.eurocopter.mgs.presentation.status.GenericStatus;
import com.eurocopter.mgs.presentation.utils.DeviationWarningUtils;
import com.eurocopter.mgs.presentation.utils.RtbUtils;
import com.eurocopter.mgs.service.aircraft.api.AircraftManagementResult;
import com.eurocopter.mgs.service.aircraft.dto.AircraftDto;
import com.eurocopter.mgs.service.aircraft.exception.AircraftAlreadyExistsException;
import com.eurocopter.mgs.service.aircraft.exception.AircraftConcurrentAccessException;
import com.eurocopter.mgs.service.exception.ConcurrentAccessException;
import com.eurocopter.mgs.service.exception.EntityNotFoundException;
import com.eurocopter.mgs.service.exception.ServiceException;
import com.eurocopter.mgs.service.rtb.dto.MainBladeTuningDto;
import com.eurocopter.mgs.service.rtb.dto.MainRotorTuningDto;
import com.eurocopter.mgs.service.rtb.dto.RtbTuningsDto;
import com.eurocopter.mgs.service.rtb.dto.TailBladeTuningDto;
import com.eurocopter.mgs.service.rtb.dto.TailRotorTuningDto;
import com.eurocopter.mgs.service.rtb.dto.VibrationDto;

/**
 * Aircraft Creation .
 * <P>
 * Action to handle when clicking on OK to create a new aircraft.
 * <P>
 */
public final class OkCreateUpdateListener extends SelectionAdapter {

   private static final Logger LOGGER = LogManager.getLogger(OkCreateUpdateListener.class);

   private static final String PAIR_ALREADY_EXISTS_TYPE_ID = Messages.CORE.getString("entered_pair_already_exist");

   private static final String PAIR_ALREADY_EXISTS_TYPE_SERIAL_NUMBER = Messages.CORE.getString("entered_pair_already_exist_type_sn");

   private static final String AIRCRAFTCONFIGURATION_PROBLEM_WITH_REGISTERNUMBER = Messages.CORE.getString("AircraftConfiguration_problem_with_registerNumber");

   /** Aircraft details dialog. */
   private final AircraftDetailsDialog aircraftDetailsDialog;

   public OkCreateUpdateListener(AircraftDetailsDialog aircraftDetailsDialog) {
      super();
      this.aircraftDetailsDialog = aircraftDetailsDialog;
   }

   @Override
   public void widgetSelected(SelectionEvent evt) {
      AircraftDetailsComposite acDetailsComposite = aircraftDetailsDialog.getAircraftDetailsComposite();
      String acTypeName = acDetailsComposite.getAircraftModel().getAircraftType().getName();
      String registerNumber = acDetailsComposite.getAircraftModel().getRegisterNumber();
      DataTransferType dataTransferType = acDetailsComposite.getAircraftModel().getDataTransferType();
      String aircraftTechnicalId = null;
      if (acDetailsComposite.getAircraftModel().getAircraft() != null) {
         aircraftTechnicalId = acDetailsComposite.getAircraftModel().getAircraft().getId();
      }
      String aircraftId = null;
      if (acDetailsComposite.getAircraftModel().getAircraftId() != null && !acDetailsComposite.getAircraftModel().getAircraftId().isEmpty()
            && !acDetailsComposite.getAircraftModel().getAircraftId().isBlank()) {
         aircraftId = acDetailsComposite.getAircraftModel().getAircraftId();
      }
      String aircraftSerialNumber = null;
      if (acDetailsComposite.getAircraftModel().getAircraftSerialNumber() != null && !acDetailsComposite.getAircraftModel().getAircraftSerialNumber().isEmpty()
            && !acDetailsComposite.getAircraftModel().getAircraftSerialNumber().isBlank()) {
         aircraftSerialNumber = acDetailsComposite.getAircraftModel().getAircraftSerialNumber();
      }
      if (validateAircraftInfos(aircraftTechnicalId, acTypeName, aircraftId, aircraftSerialNumber, registerNumber)) {
         AircraftDto acDto = new AircraftDto();
         if (aircraftTechnicalId != null) {
            acDto.setId(aircraftTechnicalId);
         }
         acDto.setAircraftType(acTypeName);
         acDto.setAvionicVersion(acDetailsComposite.getAircraftModel().getAvionicVersion().getId());
         if (acDetailsComposite.getAircraftModel().getDmauVersion() != null) {
            acDto.setDmauVersion(acDetailsComposite.getAircraftModel().getDmauVersion().getId());
         }
         acDto.setDescription(acDetailsComposite.getAircraftModel().getDescription());
         acDto.setRegisterNumber(registerNumber);
         acDto.setAircraftId(aircraftId);
         acDto.setAircraftSerialNumber(aircraftSerialNumber);
         acDto.setDataTransferType(dataTransferType);

         // Get ConfigurationDto parameter
         ConfigurationDto acConfigDto = acDetailsComposite.getAircraftConfigurationModel();
         if (acConfigDto == null) {
            acConfigDto = new ConfigurationDto();
         }

         // BCR-6605 importing a health or usage configuration closes every deviation of the aircraft.
         // The ids are read HERE, before the update: updateAircraft builds a new indicator
         // configuration set, so afterwards the active-deviation lookup no longer returns these rows
         // and nothing would ever be acknowledged.
         final boolean configurationImported = AircraftDetailsDialog.isUpdate()
               && aircraftTechnicalId != null
```

```
            && (acConfigDto.getHealthConfigurationFile() != null || acConfigDto.getUsageConfigurationFile() != null);
         final List<String> deviationsToAcknowledge;
         if (configurationImported) {
            deviationsToAcknowledge = DeviationWarningUtils.getActiveDeviationIds(aircraftTechnicalId);
         } else {
            deviationsToAcknowledge = new ArrayList<>();
         }
         if (!deviationsToAcknowledge.isEmpty()) {
            MessageDialog dialog = new MessageDialog(aircraftDetailsDialog.getShell(),
                  Messages.CORE.getString("deviationConfigUpdateConfirmTitle"), null,
                  Messages.CORE.getString("deviationConfigUpdateConfirm"), MessageDialog.WARNING,
                  new String[] { Messages.CORE.getMnemonicEscapedString("Cancel"), Messages.CORE.getString("deviationContinue") }, 0);
            // index 1 is Continue. Cancel, Escape and closing the popup all abort and do nothing.
            if (dialog.open() != 1) {
               return;
            }
         }

         // Validate configuration files

         // create RtbConfigurationDto parameter
         final RtbTuningsDto rtbDto = createRtbDto(acDetailsComposite);

         final BundleContext aircraftMangementBundleContext = FrameworkUtil.getBundle(IAircraftManagementBusiness.class).getBundleContext();
         final IAircraftManagementBusiness aircraftManagementBusiness = aircraftMangementBundleContext
               .getService(aircraftMangementBundleContext.getServiceReference(IAircraftManagementBusiness.class));

         final AircraftManagementResult aircraftManagementResult;
         try {
            if (AircraftDetailsDialog.isUpdate()) {
               Flight selectedFlight = acDetailsComposite.getSelectedFlight();
               final String applicableFlightId;
               if (selectedFlight != null) {
                  applicableFlightId = selectedFlight.getId();
               } else {
                  applicableFlightId = null;
               }
               aircraftManagementResult = aircraftManagementBusiness.updateAircraft(acDto, acConfigDto, rtbDto, applicableFlightId);
            } else {
               aircraftManagementResult = aircraftManagementBusiness.createAircraft(acDto, acConfigDto, rtbDto);
            }

            if (!deviationsToAcknowledge.isEmpty()) {
               DeviationWarningUtils.acknowledge(aircraftTechnicalId, deviationsToAcknowledge);
            }
            aircraftDetailsDialog.close();

            if (aircraftManagementResult.hasWarnings()) {
               handleAircraftConfigurationWarnings(aircraftManagementResult);
            }

         } catch (AircraftAlreadyExistsException e) {
            LOGGER.warn("Aircraft already exists", e);
            // only happens on create
            StatusManager.getManager().handle(new GenericStatus(IStatus.ERROR, Activator.PLUGIN_ID, Messages.CORE.getString("create.aborted.alreadyExists")),
StatusManager.BLOCK); //$NON-NLS-1$
            validateAircraftInfos(aircraftTechnicalId, acTypeName, aircraftId, aircraftSerialNumber, registerNumber);
         } catch (EntityNotFoundException e) {
            LOGGER.warn("Entity not found", e);
            StatusManager.getManager().handle(
                  new GenericStatus(IStatus.ERROR, Activator.PLUGIN_ID, Messages.CORE.getString("createUpdate.aborted.missingElement"), e), StatusManager.BLOCK); //$NON-NLS-1$
            validateAircraftInfos(aircraftTechnicalId, acTypeName, aircraftId, aircraftSerialNumber, registerNumber);
         } catch (AircraftConcurrentAccessException e) {
            LOGGER.warn("Aircraft concurrent access", e);
            // only happens on update
            StatusManager.getManager()
                  .handle(new GenericStatus(IStatus.ERROR, Activator.PLUGIN_ID, MessageFormat.format(Messages.CORE.getString("update.aborted.concurrent" +
e.getAircraftLock().getLockCause()), //$NON-NLS-1$
                        e.getAircraft().getRegisterNumber())),
                        StatusManager.BLOCK);
            aircraftDetailsDialog.close();
         } catch (ConcurrentAccessException e) {
            LOGGER.warn("Concurrent access", e);
            // only happens on update
            StatusManager.getManager().handle(new GenericStatus(IStatus.ERROR, Activator.PLUGIN_ID, Messages.CORE.getString("update.aborted.concurrentUpdate")),
StatusManager.BLOCK); //$NON-NLS-1$
            aircraftDetailsDialog.close();
         } catch (ServiceException e) {
            StatusManager.getManager()
                  .handle(
                        new GenericStatus(IStatus.ERROR, Activator.PLUGIN_ID,
                              MessageFormat.format(Messages.CORE.getString("create.aborted.unknownCause"), acTypeName, aircraftId, registerNumber), e), //$NON-NLS-1$
                        StatusManager.BLOCK);
            validateAircraftInfos(aircraftTechnicalId, acTypeName, aircraftId, aircraftSerialNumber, registerNumber);
         }

      }
   }

   private void handleAircraftConfigurationWarnings(AircraftManagementResult aircraftManagementResult) {
      String featureWithWarnings = "";

      if (aircraftManagementResult.getFeaturesWithWarnings().contains(FeatureName.HEALTH_MONITORING)
            || aircraftManagementResult.getFeaturesWithWarnings().contains(FeatureName.HEALTH_SIGNAL_ANALYSIS)) {
         featureWithWarnings = formatAndAddFeatureNameForWarningMessage(featureWithWarnings, "HealthConf");
      }
      if (aircraftManagementResult.getFeaturesWithWarnings().contains(FeatureName.COUNTER_AND_CYCLE)
            || aircraftManagementResult.getFeaturesWithWarnings().contains(FeatureName.EXCEEDANCE)
            || aircraftManagementResult.getFeaturesWithWarnings().contains(FeatureName.ENGINE_POWER_CHECKS)) {
         featureWithWarnings = formatAndAddFeatureNameForWarningMessage(featureWithWarnings, "UsageConf");
      }
      if (aircraftManagementResult.getFeaturesWithWarnings().contains(FeatureName.SYSTEM_FAILURE_MANAGEMENT)) {
         featureWithWarnings = formatAndAddFeatureNameForWarningMessage(featureWithWarnings, "FailureDescriptionConf");
      }
      if (aircraftManagementResult.getFeaturesWithWarnings().contains(FeatureName.DMAU_FAILURES)) {
         featureWithWarnings = formatAndAddFeatureNameForWarningMessage(featureWithWarnings, "SystemStatusConf");
      }
      if (aircraftManagementResult.getFeaturesWithWarnings().contains(FeatureName.VPU_FAILURES)) {
         featureWithWarnings = formatAndAddFeatureNameForWarningMessage(featureWithWarnings, "FailureDescriptionConf");
      }
      if (aircraftManagementResult.getFeaturesWithWarnings().contains(FeatureName.LOCAL_DIAGNOSIS)) {
         featureWithWarnings = formatAndAddFeatureNameForWarningMessage(featureWithWarnings, "LocalDiagnosisConf");
      }
      if (aircraftManagementResult.getFeaturesWithWarnings().contains(FeatureName.FAILURE_CONTEXT_MONITORING)) {
         featureWithWarnings = formatAndAddFeatureNameForWarningMessage(featureWithWarnings, "FailureContextMonitoringConf");
      }
      if (aircraftManagementResult.getFeaturesWithWarnings().contains(FeatureName.FDCR)) {
         featureWithWarnings = formatAndAddFeatureNameForWarningMessage(featureWithWarnings, "FDCRConfiguration");
      }
      if (aircraftManagementResult.getFeaturesWithWarnings().contains(FeatureName.ROTOR_TRACK_BALANCE)) {
         featureWithWarnings = formatAndAddFeatureNameForWarningMessage(featureWithWarnings, "RTBParametersConf");
      }

      StatusManager.getManager()
            .handle(
                  new GenericStatus(IStatus.WARNING, Activator.PLUGIN_ID,
                        MessageFormat.format(Messages.CORE.getString("AircraftManagement_configurationWarnings"), featureWithWarnings)),
                  StatusManager.BLOCK);
   }
```

```
   private String formatAndAddFeatureNameForWarningMessage(String message, String coreMessageKey) {
      return message + "- " + Messages.CORE.getString(coreMessageKey) + "\n";
   }

   private RtbTuningsDto createRtbDto(AircraftDetailsComposite acDetailsComposite) {
      final RtbTuningsDto rtbDto;
      if (acDetailsComposite.isValidRotorParameters()) {
         rtbDto = new RtbTuningsDto();
         MainRotorTuning mainRotorTuning = acDetailsComposite.getMainRotorTuning();
         TailRotorTuning tailRotorTuning = acDetailsComposite.getTailRotorTuning();

         final MainRotorTuningDto mainRotorTuningDto = (MainRotorTuningDto) RtbUtils.toDto(mainRotorTuning);

         for (MainTuningValue mainTuningValue : mainRotorTuning.getTuningValues()) {
            MainBladeTuningDto bladeTuningDto = new MainBladeTuningDto();
            bladeTuningDto.setBlade(mainTuningValue.getPointName());
            bladeTuningDto.setSleeveWeight(mainTuningValue.getSleeveWeight());
            bladeTuningDto.setPitchLinkLength(mainTuningValue.getPitchLinkLength());
            bladeTuningDto.setTabDeviation(mainTuningValue.getTabDeviation());
            mainRotorTuningDto.getTuningValues().add(bladeTuningDto);
         }
         VibrationDto vibrationDto = new VibrationDto();
         if (mainRotorTuning.getVibration() != null) {
            vibrationDto.setValues(mainRotorTuning.getVibration().getValues());
         }
         mainRotorTuningDto.setVibration(vibrationDto);

         rtbDto.setMainRotorTuning(mainRotorTuningDto);

         final TailRotorTuningDto tailRotorTuningDto = (TailRotorTuningDto) RtbUtils.toDto(tailRotorTuning);
         for (TailTuningValue tailTuningValue : tailRotorTuning.getTuningValues()) {
            TailBladeTuningDto bladeTuningDto = new TailBladeTuningDto();
            bladeTuningDto.setBlade(tailTuningValue.getPointName());
            bladeTuningDto.setSleeveWeight(tailTuningValue.getSleeveWeight());
            tailRotorTuningDto.getTuningValues().add(bladeTuningDto);
         }

         vibrationDto = new VibrationDto();
         if (tailRotorTuning.getVibration() != null) {
            vibrationDto.setValues(tailRotorTuning.getVibration().getValues());
         }
         tailRotorTuningDto.setVibration(vibrationDto);

         rtbDto.setTailRotorTuning(tailRotorTuningDto);
      } else {
         rtbDto = null;
      }
      return rtbDto;
   }

   private boolean validateAircraftInfos(String aircraftTechnicalId, String acTypeName, String aircraftId, String aircraftSerialNumber, String registerNumber) {
      final BundleContext aircraftBundleContext = FrameworkUtil.getBundle(IAircraftBusiness.class).getBundleContext();
      final IAircraftBusiness aircraftBusiness = aircraftBundleContext.getService(aircraftBundleContext.getServiceReference(IAircraftBusiness.class));
      boolean valid = true;

      String aircraftTypeTooltipText = "";
      if (aircraftSerialNumber != null) {
         Optional<Aircraft> aircraftBySerialNumber = aircraftBusiness.getAircraftByAircraftSerialNumber(acTypeName, aircraftSerialNumber);
         if (aircraftBySerialNumber.isPresent()) {
            Aircraft aircraft = aircraftBySerialNumber.get();
            if (!AircraftDetailsDialog.isUpdate() || (AircraftDetailsDialog.isUpdate() && !aircraft.getId().equals(aircraftTechnicalId))) {
               aircraftDetailsDialog.getAircraftDetailsComposite().getAircraftTypeDecorator().show();
               aircraftTypeTooltipText = PAIR_ALREADY_EXISTS_TYPE_SERIAL_NUMBER;
               aircraftDetailsDialog.getAircraftDetailsComposite().getAircraftTypeDecorator().setDescriptionText(aircraftTypeTooltipText);
               aircraftDetailsDialog.getAircraftDetailsComposite().getAircraftSerialNumberDecorator().show();
               aircraftDetailsDialog.getAircraftDetailsComposite().getAircraftSerialNumberDecorator().setDescriptionText(PAIR_ALREADY_EXISTS_TYPE_SERIAL_NUMBER);
               valid = false;
            }
         }
      }

      if (aircraftId != null) {
         Optional<Aircraft> aircraftById = aircraftBusiness.getAircraftByAircraftId(acTypeName, aircraftId);
         if (aircraftById.isPresent()) {
            Aircraft aircraft = aircraftById.get();
            if (!AircraftDetailsDialog.isUpdate() || (AircraftDetailsDialog.isUpdate() && !aircraft.getId().equals(aircraftTechnicalId))) {
               aircraftDetailsDialog.getAircraftDetailsComposite().getAircraftTypeDecorator().show();
               if (aircraftTypeTooltipText != "") {
                  aircraftTypeTooltipText = aircraftTypeTooltipText + "\n" + PAIR_ALREADY_EXISTS_TYPE_ID;
               } else {
                  aircraftTypeTooltipText = PAIR_ALREADY_EXISTS_TYPE_ID;
               }
               aircraftDetailsDialog.getAircraftDetailsComposite().getAircraftTypeDecorator().setDescriptionText(aircraftTypeTooltipText);
               aircraftDetailsDialog.getAircraftDetailsComposite().getAircraftIdDecorator().show();
               aircraftDetailsDialog.getAircraftDetailsComposite().getAircraftIdDecorator().setDescriptionText(PAIR_ALREADY_EXISTS_TYPE_ID);
               valid = false;
            }
         }
      }

      Optional<Aircraft> aircraftByRegisterNumber = aircraftBusiness.getAircraftByRegisterNumber(registerNumber);
      if (aircraftByRegisterNumber.isPresent()) {
         Aircraft aircraft = aircraftByRegisterNumber.get();
         if (!AircraftDetailsDialog.isUpdate() || (AircraftDetailsDialog.isUpdate() && !aircraft.getId().equals(aircraftTechnicalId))) {
            aircraftDetailsDialog.getAircraftDetailsComposite().getRegisterNumberDecorator().show();
            aircraftDetailsDialog.getAircraftDetailsComposite().getRegisterNumberDecorator().setDescriptionText(AIRCRAFTCONFIGURATION_PROBLEM_WITH_REGISTERNUMBER);
            valid = false;
         }
      }
      return valid;
   }
}
```

3 – the five message keys

File: the message bundle that already holds `MaxPointsPerHour` and `aircraftDetailsDialogUpdateTitle`

Without these the popups show the raw key or throw `MissingResourceException`. The text is Benoit's, word for word. `\n` gives his line breaks.

```
deviationConfigUpdateWarning=Warning : Deviations are set on this aircraft
deviationConfigImportWarning=Warning : Deviations are set on this aircraft
deviationConfigUpdateConfirmTitle=Deviations
deviationConfigUpdateConfirm=Condition Indicators in the deviation list will be acknowledged\n\nPlease open a TE on AirbusWorld > Request >
Technical Request\nselect the business Domain "DR : Discrepancy Report" before continue
deviationContinue=Continue
```

`Cancel` and `OK` are already in the bundle – do not add them again.

4 – rebuild and test

Client rebuild only. Nothing changes on the server.

#	Action	Expect
a	Aircraft **with** deviations → Update Aircraft	Red line **"Warning : Deviations are set on this aircraft"** on the popup
b	Same popup → **Configurations...**	Same red warning at the top of the Import Configurations popup
c	Pick a health file → OK → OK	Benoit's 3-line confirm popup, buttons **Cancel** / **Continue**
d	Press **Cancel**	**Nothing is done.** Aircraft not updated, deviations still in the deviation list
e	Press **Escape** on that popup	Same as Cancel – nothing is done
f	Repeat c, press **Continue**	Aircraft updated. **Deviation list empty**, every row moved to **Deviation history** with end date = today
g	Reopen the Health page	No green rows left, every field editable again. Deviations must be re-created by hand
h	Pick a **usage** file only → OK	Confirm popup appears too
i	Aircraft with **no** deviations → Update	**No** warning, **no** confirm popup. Behaviour unchanged
j	**Create** a new aircraft	No warning, no confirm popup – it is update-only

Confirm in the database after test f

```
SELECT id, te_reference, start_date, end_date
FROM public.indicator_deviation
ORDER BY start_date DESC;
```

Every row of that aircraft must carry an `end_date` of today. A row still showing `end_date` NULL means the acknowledge ran after the update again.

And in the client log

```
BCR-6605 N deviation(s) acknowledged on aircraft <id> because the configuration was updated.
```